



UNIVERSIDAD DE GRANADA

**Escuela Técnica Superior de Ingenierías Informática y
Telecomunicaciones(ETSIIT).
Grado en Ingeniería Informática.
Periféricos y Dispositivos de Interfaz Humana
Curso Académico 2025-26
Alumno:Carlos Alberto Rodríguez Ferreira**

Uso del Wiimote como Dispositivo de Entrada.

1.Introducción.....	2
2.Entorno Hardware.....	2
2.1 Tinkercad.....	2
2.2 Arduino.....	3
2.2.1 Entorno Apagado.....	4
2.2.2 Entorno Encendido.....	4
2.3 Código.....	5
3. Entorno Software.....	6
3.1 Descarga, instalación de GlovePIEy configuración inicial.....	6
3.1.1 Descarga e Instalación de GlovePIE.....	6
3.1.2 Configuración del Mando.....	7
3.1.3 Configuración inicial.....	8
3.2 Configuración de las opciones.....	8
3.2.1 Modo 1: Navegación de Escritorio.....	8
3.2.2 Modo 2: Juegos en Primera Persona(FPS).....	9
3.2.3 Modo 3: Mando Tradicional.....	9
3.3 Código de GlovePIE.....	10
4. Conclusiones.....	13

1.Introducción

Para la realización de este pequeño proyecto, voy a dividir el trabajo en secciones, siendo la primera sección la creación del entorno hardware para una correcta usabilidad del wiimote. La segunda sección será la configuración a nivel de software para hacer que sea 100% funcional el wiimote con el ordenador. Por último, la configuración y mapeo de botones del wiimote para poder usarlo en diferentes escenarios(juegos, navegar, etc).

2.Entorno Hardware

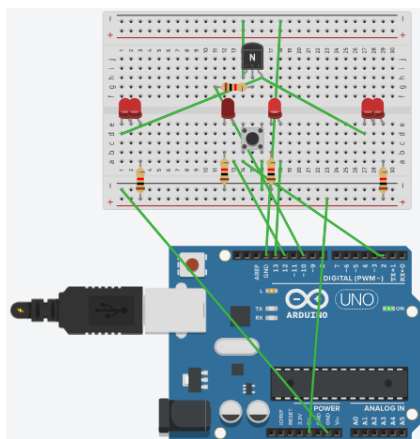
El Wiimote funciona con infrarrojos, la barrita que suele traer la consola nada mas es que infrarrojos que el mando los detecta y calcula la posición del ratón en base a las luces infrarrojas. Para poder usar el mando con todas sus funcionalidades, crearé mi propia barrita con arduino, de esa manera podré tener de forma “nativa” la barrita para el ordenador sin tener que comprar adaptadores o una barrita para el ordenador.

2.1 Tinkercad

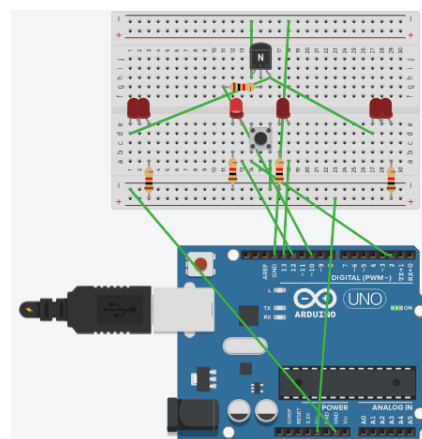
Antes de empezar a tocar con diodos, resistencias, botones, etc. Lo he montado en el tinkercad y he probado su funcionalidad. El sistema que voy a usar es simple, consiste de 4 leds infrarrojos, 1 led rojo, 1 led verde, 1 botón, 2 resistencias de 100 ohm y 2 resistencias de 220 ohm, 1 resistencia de 1kohm y un transistor npn.

La intención es hacer un “switch” cuando le doy al botón se enciende los 4 leds infrarrojos y el led verde, si le vuelvo a dar se apagaran los citados anteriormente y se encenderá el led rojo. Como se puede notar, los leds de colores sirven para poder saber si está encendido o apagado, ya que el infrarrojo no está en nuestro espectro y no podemos percibirlo.

Usaré las resistencias de 100 ohms para 2 leds infrarrojos, como están en serie la resistencia necesaria baja para unos 100 ohm aproximadamente. Las resistencias de 220 ohms son para los leds rojo y verde, estas no se encuentran en serie. y usaré un transistor npn para en resumidas cuentas poder apagar y encender los leds cuando le de al boton. Ya que para que funcionen bien los leds tengo que conectarlos a 5v.



Entorno activo o encendido



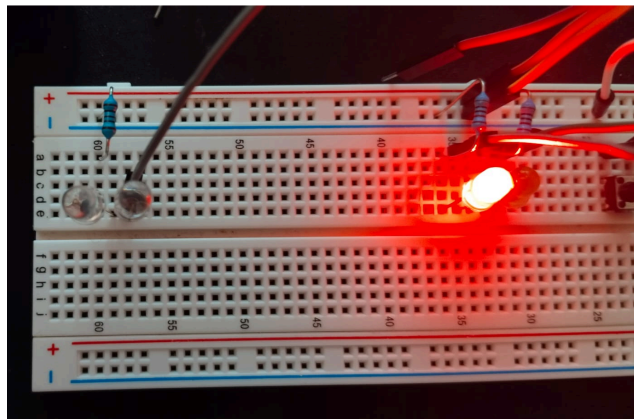
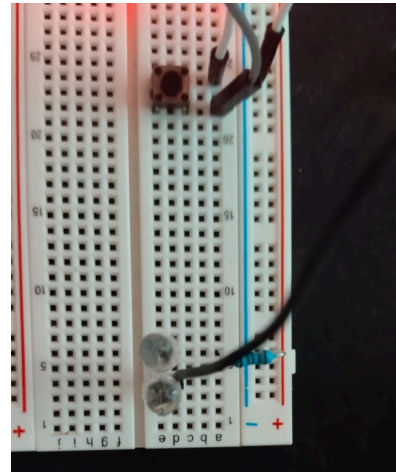
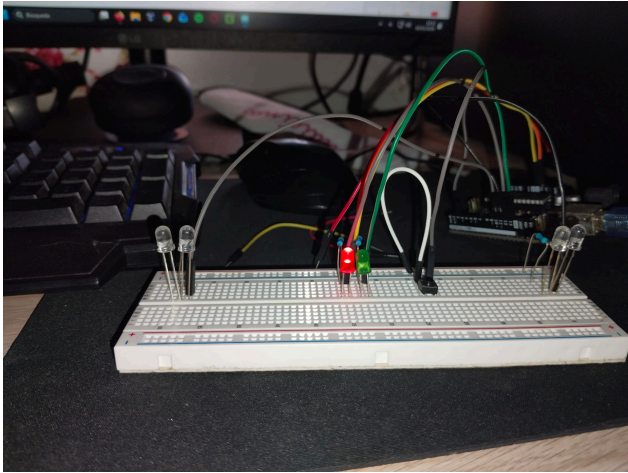
Entorno en reposo o apagado

2.2 Arduino

Una vez que ya lo he probado en el simulador toca montarlo en el kit arduino y probar si funciona. Para poder saber si el infrarrojo realmente funciona, he tenido que usar la camara del móvil, ya que este si detecta el infrarrojo y sale en pantalla una especie de lilás, ese lilás es el infrarrojo.

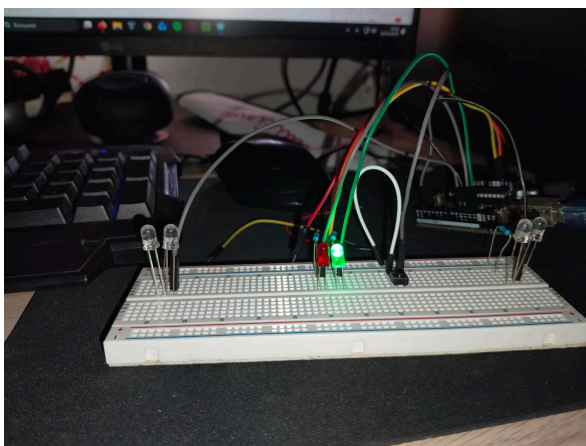
2.2.1 Entorno Apagado

Cuando está apagado el único led que emite luz es el led rojo. Los demás quedan apagados como se puede apreciar en las imágenes.



2.2.2 Entorno Encendido

Cuando está encendido el único led que no emite nada es el rojo. Los demás leds quedan encendidos como se puede apreciar en las imágenes.





2.3 Código

El código que usé es bastante simple, inicialicé cada uno de los pines, guardé un estado inicial que es apagado y cuando le doy al boton, el estado cambia de apagado a encendido y viceversa.

```
const int pinTransistor = 10; // Control del NPN
const int pinLedVerde = 13; // Indicador visual de ON
const int pinLedRojo = 12; // Indicador visual de OFF
const int pinBoton = 2; // Pin de lectura del botón físico

// Variables para la Máquina de Estados
bool estadoBarra = false; // Comienza apagada por defecto
bool estadoBotonAnterior = LOW;

void setup() {
    // Configuramos todos los actuadores como salidas
    pinMode(pinTransistor, OUTPUT);
    pinMode(pinLedVerde, OUTPUT);
    pinMode(pinLedRojo, OUTPUT);

    // Configuramos el botón como entrada con resistencia interna
    pinMode(pinBoton, INPUT_PULLUP);

    // --- ESTADO INICIAL DEL SISTEMA ---
    // Nos aseguramos de que al enchufar el Arduino, la placa refleje
    // visualmente que la barra está apagada.
    digitalWrite(pinTransistor, LOW); // IR apagado
    digitalWrite(pinLedVerde, LOW); // Verde apagado
    digitalWrite(pinLedRojo, HIGH); // Rojo encendido
}

void loop() {
    bool estadoBotonActual = digitalRead(pinBoton);
```

```

// Detectamos el "Flanco de Bajada" (pulsación exacta)
if (estadoBotonActual == LOW && estadoBotonAnterior == HIGH) {

    // Invertimos el estado lógico
    estadoBarra = !estadoBarra;

    // --- APLICAMOS EL NUEVO ESTADO AL HARDWARE ---
    if (estadoBarra == true) {
        // SISTEMA ACTIVO (ON)
        digitalWrite(pinTransistor, HIGH); // Abre el transistor (IR ON)
        digitalWrite(pinLedVerde, HIGH); // Enciende aviso Verde
        digitalWrite(pinLedRojo, LOW); // Apaga aviso Rojo

    } else {
        // SISTEMA EN REPOSO (OFF)
        digitalWrite(pinTransistor, LOW); // Cierra el transistor (IR OFF)
        digitalWrite(pinLedVerde, LOW); // Apaga aviso Verde
        digitalWrite(pinLedRojo, HIGH); // Enciende aviso Rojo
    }

    // Filtro anti-rebote (Debounce)
    delay(50);
}

// Guardamos el estado para el siguiente ciclo
estadoBotonAnterior = estadoBotonActual;
}

```

3. Entorno Software.

Una vez que tenemos el entorno hardware preparado, pasamos al software, antes de cualquier cosa, es necesario instalar el FreePIE que es un emulador de entradas programable, ese software va a ser lo que traducirá las entradas del wiimote al ordenador.

3.1 Descarga, instalación de GlovePIEy configuración inicial

3.1.1 Descarga e Instalación de GlovePIE

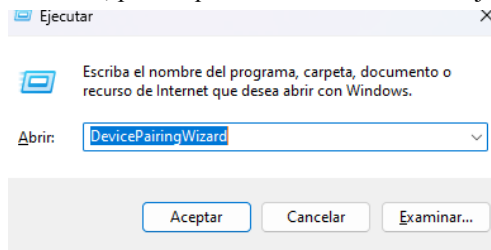
El GlovePIE es un software a día de hoy desfasado ya que está abandonado, para la mayoría de los casos hoy en día se usa el FreePIE, pero, para este caso, lo usaremos, ya que el GlovePIE si nos dá la posibilidad escanear con el mando los puntos infrarojos, cosa que el FreePIE no nos permite.

Para su descarga vamos a ir al [GitHub de GlovePIEPreservation](#) y descargaremos la versión 0.43 que es la más estable.

Este programa no es necesario instalar como tal, ya que es portable, tenemos que descomprimir y ejecutar. Antes de proseguir usando el GlovePIE es necesario emparejar el wiimote con el ordenador.

3.1.2 Configuración del Mando

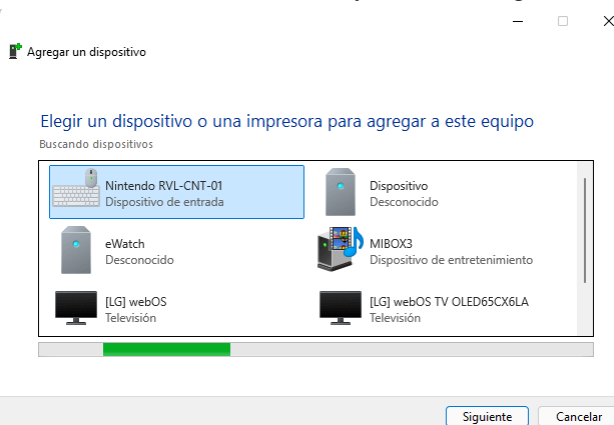
Como se dijo anteriormente, es necesario emparejarlo con el ordenador. El mando se conecta a través de bluetooth, por lo que en Windows abrimos ejecutar y pondremos “DevicePairingWizard”



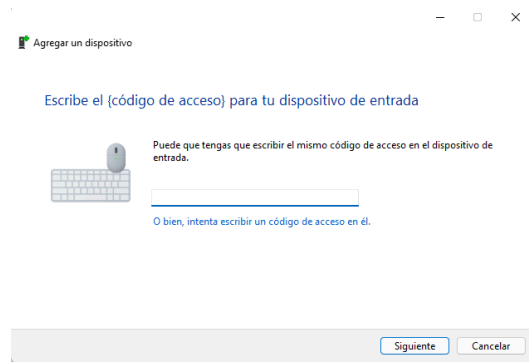
Después encenderemos el wiimote dándole al 1 y al 2 al mismo tiempo, haciendo que el mando entre en modo emparejamiento



Y al introducir el comando anterior tiene que salir la siguiente ventana con un dispositivo llamado Nintendo RVL-CNT-01. Lo seleccionamos y le damos a siguiente.



Saldrá una ventana que pedirá un pin, no se le pondrá ningún pin y le damos a siguiente



Se conectará y el mando de la wii seguirá parpadeando. Es normal, ya que cuando este se conecta a la wii deja de parpadear cuando la wii le dice el número de jugador que es, como el ordenador no le indica el número de jugador, simplemente queda parpadeando eternamente.

3.1.3 Configuración inicial

A diferencia de FreePIE que usa scripts de Python, el GlovePIE usa un lenguaje propio similar a c++. Para poder configurar el mando voy a usar la IA para que me programe las funciones que quiero implementar que son 3, siendo el primero y el determinado el modo ratón, el cual podremos usarlo para navegar, el 2º modo será el modo juego, el cual podremos usar el wiimote para jugar juegos que sea necesario puntería como juegos FPS(First Person Shooter) y el ultimo será el modo mando que vamos a poder usar los botones, pero, no podremos usar el infrarrojo. Para ir alternando entre modos elegí el botón home, ya que es un botón que en la wii te podía devolver al menú principal de la consola.

3.2 Configuración de las opciones

Hay algunos controles que son globales e independiente del modo elegido seguirán siendo los mismos, estos son los siguientes.

- ❖ Boton home
 - Cambio de modos
 - Los leds indicaran el modo en el que se encuentra
- ❖ Boton -
 - El sistema informa por voz el % de bateria

3.2.1 Modo 1: Navegación de Escritorio

Este modo se ha desarrollado para usar el wiimote como un raton en el que “apuntas” a lo que quieres. Los controles son los siguientes

- ❖ Puntero
 - Movimiento del cursor
- ❖ Boton A
 - Boton izquierdo del raton
- ❖ Boton B
 - Boton derecho del ratón
- ❖ Joystick(Nunchuk)
 - Scroll
- ❖ Botón Z(Nunchuk)
 - Teclado en Pantalla
- ❖ Boton C(Nunchuk)
 - Enter

3.2.2 Modo 2: Juegos en Primera Persona(FPS)

Este modo se ha desarrollado para poder tener una mayor inmersión en los juegos de primera persona, ya que apuntas con el mando de la wii.

- ❖ Puntero
 - Movimiento de la cámara
- ❖ Boton B
 - Gatillo(boton izquierdo del raton)
- ❖ Boton A
 - Recargar Arma(R)
- ❖ Boton 2
 - Cambio rápido de arma(Q)
- ❖ Cruceta arriba
 - Arma principal(1)
- ❖ Cruceta derecha
 - Arma secundaria(2)
- ❖ Cruceta abajo
 - Arma blanca(3)
- ❖ Cruceta izquierda
 - Granada(4)
- ❖ Boton 1
 - Armas/equipacion especial(5)
- ❖ Joystick(Nunchuk)
 - Movimiento del personaje(w,a,s,d)
- ❖ Boton C(Nunchuk)
 - Saltar(Espacio)
- ❖ Boton Z(Nunchuk)
 - Agacharse(Ctrl izquierdo)

3.2.3 Modo 3: Mando Tradicional

Este modo se ha desarrollado para poder jugar juegos que no es necesario apuntar a la pantalla, en este modo se prescinde del sensor infrarrojo

- ❖ Cruceta/Joystick
 - Movimiento del personaje(flechas de movimiento)
- ❖ Boton 2
 - Tecla X
- ❖ Boton 1
 - Tecla Z
- ❖ Boton A
 - Tecla Enter
- ❖ Boton B
 - Tecla Espacio
- ❖ Boton C
 - Tecla S
- ❖ Boton Z
 - Tecla D
- ❖ Boton
 - Tecla Esc

3.3 Código de GlovePIE

Como fue dicho anteriormente, el código ha sido desarrollado por la IA, pero, supervisado por mi para que pudiera hacer las funciones principales a la perfección.

```
// =====  
// PROYECTO: BARRA SENSORA IR + WIIMOTE (UGR)  
// Autor: Estudiante Ingeniería Informática  
// =====  
  
// --- INICIALIZACIÓN DE VARIABLES ---  
if starting then  
    var.modo = 1  
    var.sensibilidad = 0.6 // Ajustable para el modo FPS  
    var.inicializado = true  
    Wiimote.Leds = 1  
    Say("Sistema iniciado. Modo escritorio.")  
end if  
  
// --- MONITORIZACIÓN DE ESTADO CORREGIDA ---  
var.displayBat = 0  
if Wiimote.Battery > 2 then  
    var.displayBat = round(Wiimote.Battery / 2)  
else  
    var.displayBat = round(Wiimote.Battery * 100)  
end if  
if var.displayBat > 100 then var.displayBat = 100  
  
debug = "Modo: " + var.modo + " | Batería: " + var.displayBat + "% | IR: " + Wiimote.PointerVisible  
// --- SELECTOR DE MODOS (Botón Home) ---  
if pressed(Wiimote.Home) then  
    var.modo = var.modo + 1  
    if var.modo > 3 then var.modo = 1  
    Wiimote.Leds = var.modo // El número de luces indica el modo  
  
    if var.modo == 1 then Say("Modo escritorio")  
    if var.modo == 2 then Say("Modo F P S")  
    if var.modo == 3 then Say("Modo multimedia")  
end if  
  
// --- INFORME DE BATERÍA (Botón Minus) ---  
if pressed(Wiimote.Minus) then  
    // Detectamos si el valor es crudo (0-200) o normalizado (0-1)  
    if Wiimote.Battery > 2 then  
        var.porcentaje = round(Wiimote.Battery / 2)  
    else  
        var.porcentaje = round(Wiimote.Battery * 100)  
    end if  
  
    // Limitamos el máximo a 100 por si el driver da valores extraños  
    if var.porcentaje > 100 then var.porcentaje = 100
```

```

    Say("Carga de la batería al " + var.porcentaje + " por ciento")
end if

// =====
// MODO 1: RATÓN DE ESCRITORIO (WINDOWS)
// =====

if var.modos == 1 then
    // Movimiento IR suavizado
    if Wiimote.PointerVisible then
        Mouse.x = Smooth(Wiimote.PointerX, 12)
        Mouse.y = Smooth(Wiimote.PointerY, 12)
    end if

    // Botones principales
    Mouse.LeftButton = Wiimote.A
    Mouse.RightButton = Wiimote.B

    // Scroll con Joystick (Nunchuk)
    if Wiimote.Nunchuk.JoyY < -0.5 then
        Mouse.WheelUp = true
        wait 60ms
        Mouse.WheelUp = false
    end if
    if Wiimote.Nunchuk.JoyY > 0.5 then
        Mouse.WheelDown = true
        wait 60ms
        Mouse.WheelDown = false
    end if

    // Teclado Virtual (Combinación Win+Ctrl+O para Teclado en pantalla)
    if pressed(Wiimote.Nunchuk.ZButton) then
        Press(Key.Windows)
        Press(Key.LeftCtrl)
        Press(Key.O)
        wait 50ms
        Release(Key.O)
        Release(Key.LeftCtrl)
        Release(Key.Windows)
    end if

    Key.Enter = Wiimote.Nunchuk.CButton
end if

// =====
// MODO 2: FPS (ASSUALTCUBE / WOLFENSTEIN / COD)
// =====

if var.modos == 2 then
    // Lógica de cámara (Movimiento Relativo/DirectInput)
    if Wiimote.PointerVisible then
        var.diffX = (Wiimote.PointerX - 0.5) * 80
        var.diffY = (Wiimote.PointerY - 0.5) * 80
        Mouse.DirectInputX = Mouse.DirectInputX + (Smooth(var.diffX, 8) * var.sensibilidad)
    end if
end if

```

```

    Mouse.DirectInputY = Mouse.DirectInputY + (Smooth(var.diffY, 8) * var.sensibilidad)
end if

// Combate y Acciones
Mouse.LeftButton = Wiimote.B // Disparar (Gatillo)
Key.R = Wiimote.A // Recargar
Key.Q = Wiimote.Two // Cambio rápido de arma

// Selector de Armas (1-5)
Key.One = Wiimote.Up
Key.Two = Wiimote.Right
Key.Three = Wiimote.Down
Key.Four = Wiimote.Left
Key.Five = Wiimote.One

// Movimiento (Nunchuk)
Key.W = Wiimote.Nunchuk.JoyY < -0.3
Key.S = Wiimote.Nunchuk.JoyY > 0.3
Key.A = Wiimote.Nunchuk.JoyX < -0.3
Key.D = Wiimote.Nunchuk.JoyX > 0.3

Key.Space = Wiimote.Nunchuk.CButton // Saltar
Key.LeftControl = Wiimote.Nunchuk.ZButton // Agacharse
end if

// =====
// MODO 3: EMULACIÓN Y GAMING 2D
// =====
if var.modos == 3 then
    // Movimiento (Digital + Analógico)
    Key.Up = Wiimote.Up or (Wiimote.Nunchuk.JoyY < -0.4)
    Key.Down = Wiimote.Down or (Wiimote.Nunchuk.JoyY > 0.4)
    Key.Left = Wiimote.Left or (Wiimote.Nunchuk.JoyX < -0.4)
    Key.Right = Wiimote.Right or (Wiimote.Nunchuk.JoyX > 0.4)

    // Botones de Acción
    Key.X = Wiimote.Two // Botón 2 -> Acción A (Salto)
    Key.Z = Wiimote.One // Botón 1 -> Acción B (Ataque)

    // Gestión de Menús y Sistema
    Key.Enter = Wiimote.A // Botón A -> Start
    Key.Space = Wiimote.B // Botón B -> Select
    Key.Escape = Wiimote.Plus // Botón Plus -> Menú Principal / Salir

    // Botones Auxiliares (Nunchuk)
    Key.S = Wiimote.Nunchuk.CButton // Botón C -> L (Hombro izquierdo)
    Key.D = Wiimote.Nunchuk.ZButton // Botón Z -> R (Hombro derecho)
end if

```

4. Conclusiones

Ha sido un proyecto bastante interesante de desarrollar, quise implementar un poco de lo que hicimos en clase que fue trabajar con arduino. Además que siempre quise configurar el mando de la wii para poder jugar en el ordenador y usarlo como un ratón. También quería usar el altavoz del mando, pero, al usarlo la red bluetooth queda saturada y no funciona bien el mando mientras se reproduce el sonido.

Este es la parte mayor por así decirlo de este proyecto, voy a seguir investigando cosas e ir intentando mejorar funciones o programas para dejar lo mejor posible el wiimote con el ordenador hasta el día de la presentación.